

OVERSEE Architecture Deep-Dive Manual Using COMP-001

Version baseline: v0.6.2

Main explanatory case: COMP-001

Scope: input information, layer-by-layer processing, generated files and final OVERSEE output

Author: F. Requena-Alcaraz

1. Purpose of this manual

This manual explains the OVERSEE executable architecture in detail.

It uses COMP-001 as the main explanatory case because it is the paper-aligned compressor scenario and the easiest case to present first. However, the same executable architecture applies to all 20 scenarios included in the v0.6.2 workbench.

The goal is to make clear:

1. Where the input information comes from.
2. How the information enters OVERSEE.
3. What each layer does.
4. What each generated file stores.
5. What remains as the final output of OVERSEE.
6. Why the final recommendation is not just a predictive alert.

This document is written for a technically aware maintenance or operations audience, not only for programmers.

2. The key idea

A predictive alert is not a maintenance decision.

A predictive model may say:

The compressor has a bearing degradation risk.

But a maintenance decision also needs to know:

```
Is the evidence valid?  
How critical is the asset?  
What is the condition trend?  
What happened in previous interventions?  
Is production under pressure?  
Is there a downtime window?  
Are spare parts available?  
Is a technician available?  
Is human review mandatory?  
Can the final recommendation be audited?
```

OVERSEE exists to bridge this gap.

It transforms:

```
predictive signal
```

into:

```
governed recommendation package
```

through five layers.

3. COMP-001 case overview

COMP-001 represents an industrial air compressor with bearing degradation risk.

The scenario is stored as:

```
demo/interactive_walkthrough/scenarios/comp_001_default.json
```

This scenario contains two kinds of information:

1. Narrative information for the walkthrough.
2. Executable information for the real runner.

The narrative part includes:

```
layer_inputs  
expected_layer_outputs
```

The executable part includes:

```
executable_inputs.alert  
executable_inputs.raw_sensor_context  
executable_inputs.requested_context  
executable_inputs.enterprise_sources
```

4. Where the input information comes from

In the demo, there are no live connections to real ERP, CMMS, historian or inventory systems.

Instead, each scenario contains a controlled synthetic enterprise context in its JSON file.

For COMP-001, this synthetic enterprise context represents the type of information that would normally come from real enterprise systems.

4.1 Predictive alert

Source represented:

```
PdM analytics service
```

Information represented:

```
alert_id  
asset_id  
asset_type  
line_id  
suspected_failure_mode  
alert_type  
predictive_horizon_hours  
confidence_score  
model_name  
model_version  
triggered_at
```

Stored in:

```
executable_inputs.alert
```

4.2 Raw sensor context

Source represented:

Condition monitoring / historian

Information represented:

vibration trend
temperature trend
pressure trend
alarm count
last sensor timestamp
possible data quality flags

Stored in:

`executable_inputs.raw_sensor_context`

4.3 Requested context

This tells the runner which business context should be requested.

Information represented:

maintenance history lookback period
production context horizon
include_inventory_and_resources
include_policy_governance

Stored in:

`executable_inputs.requested_context`

4.4 Enterprise sources

These are scenario-backed equivalents of enterprise systems.

Stored in:

`executable_inputs.enterprise_sources`

They include:

```
asset_metadata  
maintenance_history  
operational_context  
inventory_and_resources  
policy_governance
```

This is important because the decision is not based only on the predictive alert. It is based on an enriched enterprise context.

5. How the executable runner uses the scenario

The direct runner is:

```
$env:PYTHONPATH = "."  
python scripts\run_scenario_all_layers_demo.py --scenario COMP-001
```

The runner performs this sequence:

1. Load the COMP-001 scenario JSON.
2. Build a predictive alert request from executable_inputs.alert.
3. Create a scenario-backed enterprise API client.
4. Execute Layer 1.
5. Execute Layer 2.
6. Execute Layer 3.
7. Execute Layer 4.
8. Execute Layer 5.
9. Write all generated artifacts to an output folder.
10. Return a concise execution result.

The output folder follows this pattern:

```
outputs/scenario_all_layers_comp_001_YYYYMMDD_HHMMSS/
```

6. Layer 0 - Scenario and request preparation

This is not an OVERSEE layer, but it is important for understanding the demo.

6.1 00_scenario.json

Stores the complete scenario definition used for the run.

It includes:

```
scenario_id
title
description
asset_id
asset_type
failure_mode
master_case flag
decision_pattern
layer_inputs
expected_layer_outputs
executable_inputs
```

Why it matters:

It is the source of truth for the scenario execution.

6.2 00_predictive_alert_request.json

Stores the executable alert request created from the scenario.

It includes:

```
alert identity
asset identity
failure mode
alert type
predictive horizon
confidence
model metadata
timestamp
```

Why it matters:

It shows exactly what predictive signal is being injected into OVERSEE.

7. Layer 1 - Evidence intake, aggregation and validation

Simple explanation:

Layer 1 builds the evidence folder.

Layer 1 receives the predictive alert and requests supporting evidence from the scenario-backed enterprise APIs.

7.1 `01_received_predictive_alert.json`

Stores the alert after Layer 1 receives it.

Why it matters:

It preserves the original predictive signal received by the system.

7.2 `01_enterprise_api_calls.json`

Stores the simulated enterprise API calls performed by the scenario-backed client.

It shows retrieved information such as:

```
asset metadata
maintenance history
operational context
inventory and resources
policy governance
```

Why it matters:

It proves that the decision is not based only on the alert.

7.3 `01_aggregated_evidence_package.json`

Stores the consolidated evidence package.

It brings together:

```
predictive alert
condition trends
asset criticality
maintenance history
production context
resource context
policy context
provenance
```

Why it matters:

It is the first complete evidence view of the case.

7.4 01_validation_report.json

Stores whether the evidence package is valid.

It may include:

```
valid true/false  
missing sources  
payloads with data quality flags  
validation notes
```

For COMP-001, the expected behaviour is:

```
valid = true
```

For DATA-001, the expected behaviour is different:

```
valid = false  
diagnostic_review
```

Why it matters:

It allows OVERSEE to stop or redirect the flow when evidence quality is not good enough.

8. Layer 2 - Contextualization

Simple explanation:

Layer 2 turns raw evidence into operational context.

8.1 02_canonical_case_context.json

Stores the normalized case context used by downstream layers.

It typically contains:


```
case identity
asset context
risk context
operational context
maintenance resources
policy requirements
data quality flags
```

Why it matters:

It creates a common language for the rest of OVERSEE.

8.2 02_contextualization_rule_trace.json

Stores the trace of contextualization rules.

It may include reasoning about:

```
urgency
production pressure
resource feasibility
human review requirement
data quality concerns
```

Why it matters:

It makes contextualization explainable.

8.3 02_layer2_contextualization_result.json

Stores the final Layer 2 output.

For COMP-001, the expected meaning is:

```
high urgency
near downtime window
intervention feasible
review required
```

Why it matters:

It explains why the case is operationally important and feasible.

9. Layer 3 - Case lifecycle management

Simple explanation:

Layer 3 checks whether the organization is ready to move.

9.1 `03_case_lifecycle_trace.json`

Stores lifecycle events such as:

```
case opened
evidence package received
contextualization completed
tasks activated
milestones reached
blockers identified
decision readiness assessed
```

Why it matters:

It shows how the case evolved before a decision was produced.

9.2 `03_case_management_state.json`

Stores the current state of the case.

It typically includes:

```
case status
case lifecycle stage
active tasks
milestones
blockers
decision readiness
human review status
```

For `COMP-001`, the expected state is:

```
decision-ready case
open
tasks active
milestones reached
no blockers
```

For `DATA-001`, the state is different:

```
evidence_review  
diagnostic clarification required
```

Why it matters:

It prevents OVERSEE from treating every alert as ready for execution.

10. Layer 4 - Decision logic and recommendation formulation

Simple explanation:

Layer 4 converts context into a governed recommendation.

10.1 `04_dmn_decision_evaluation.json`

Stores the DMN-like rule evaluation.

It includes outputs such as:

```
priority  
execution mode  
intervention feasibility  
human review requirement  
rule rationale  
triggered status  
input fields  
output fields
```

For `COMP-001`, the expected result is:

```
priority = high  
execution_mode = controlled_planning  
human_review_required = true
```

Why it matters:

It makes the recommendation rule-driven and inspectable.

10.2 04_recommendation_path_outputs.json

Stores the recommendation-path output.

It may include:

```
final priority
recommended execution mode
recommendation rationale
review requirements
handoff readiness
```

Why it matters:

It acts as the bridge between decision logic and final packaging.

11. Layer 5 - Governed packaging and traceability

Simple explanation:

Layer 5 makes the recommendation auditable and ready for handoff.

11.1 05_governed_recommendation_package.json

This is the main final output of OVERSEE.

It packages:

```
case summary
evidence references
decision output
recommendation
human review requirement
workflow handoff proposal
governance metadata
```

For COMP-001, the final package represents:

```
high priority
controlled planning
human review retained
work-order proposal ready
```

Why it matters:

This is the final governed recommendation package.

11.2 05_traceability_index.json

Stores traceability links across the run.

It helps answer:

Which evidence supported the recommendation?
Which context was used?
Which rules were evaluated?
Which files prove the decision path?

Why it matters:

It makes the recommendation auditable.

11.3 05_execution_manifest.json

Stores execution metadata.

It may include:

scenario id
case id
timestamps
generated files
execution path
configuration metadata

Why it matters:

It records how the run was executed.

11.4 05_scenario_execution_summary.md

This is a human-readable summary of the scenario execution.

Why it matters:

It is the easiest artifact to read first after a scenario run.

12. Final OVERSEE output

The most important final output is:

```
05_governed_recommendation_package.json
```

But the real value is the complete governed evidence chain:

```
00_scenario.json  
00_predictive_alert_request.json  
01_aggregated_evidence_package.json  
01_validation_report.json  
02_canonical_case_context.json  
03_case_management_state.json  
04_dmn_decision_evaluation.json  
05_governed_recommendation_package.json  
05_traceability_index.json  
05_scenario_execution_summary.md
```

Together, these files show:

```
what came in  
how it was interpreted  
what decisions were made  
why they were made  
what output was produced  
how it can be audited
```

13. Why this works for all 20 cases

All 20 scenarios follow the same executable structure:

```
scenario JSON  
-> executable inputs  
-> scenario-backed enterprise APIs  
-> Layer 1 evidence package  
-> Layer 2 context  
-> Layer 3 case state  
-> Layer 4 decision logic  
-> Layer 5 governed package
```

The difference is not the architecture.

The difference is the scenario content:

```
asset type  
failure mode  
risk horizon  
confidence  
condition trend  
production pressure  
resource availability  
criticality  
policy requirements  
data quality flags
```

That is why the system can produce different behaviours:

```
controlled_planning  
standard_planning  
constrained_execution  
diagnostic_review
```

14. Practical demo message

The best way to explain the demo is:

```
COMP-001 shows the normal controlled-planning path.  
PUMP-001 shows that risk can be high but execution constrained.  
DATA-001 shows that questionable evidence triggers diagnostic review instead of blind action.
```

This demonstrates that OVERSEE does not copy the same answer across cases.

It applies the same governed architecture to different industrial contexts.

15. Closing explanation

OVERSEE is not only a predictive maintenance model.

It is a governed prediction-to-decision orchestration artifact.

It shows how industrial evidence, enterprise context, lifecycle state, explicit rules, human review and traceability can be combined before maintenance action is recommended.